

SmartCare — UI documentation by actor

Generated for the CMA codebase. Describes which PHP templates run first, which CSS loads in order, and what is shared between roles.

How every page is reached

1. Browser requests `public/index.php` → `app/init.php` → router in `app/core/App.php` (`?url=...`).
2. Matched controller method runs and includes a view via `$this->view(...)` (or landing's own loader).
3. The view and any `include / require` templates output HTML and `<link rel="stylesheet">` tags. There is no automatic CSS bundler for the whole app.

Shared design-system bundle (Admin + HR + Client)

File: `app/views/templates/admin/ad_admin_core_styles.php`

When included, stylesheets load in this order:

1. Google Fonts — Inter
2. Font Awesome 6.5 (CDN)
3. Boxicons (CDN)
4. `public/css/common/sidebar-badges.css`
5. `public/css/admin/ad_header.css`
6. `public/css/admin/ad_sidebar.css`
7. `public/css/admin/admin-ui.css` (main shared UI tokens, tables, cards, forms)
8. Deferred script: `public/js/admin/admin-form-validation.js`

HR and Client shells include this file from their layout heads so chrome matches Admin.

1) Public — Landing / marketing

Entry: default route → `LandingController::home()` → `app/views/landing/home.php` .

CSS: mainly `public/css/landing.css` ; extra fonts / hero bits may be inline in `home.php` .

Shared admin bundle: No (`ad_admin_core_styles.php` is not used).

2) Guest — Login

View: `app/views/auth/login.php` .

CSS: `public/css/login.css` , Google Fonts (Poppins), Boxicons. Paths in file may use `/CMA/public/...` (hard-coded).

Shared admin bundle: No.

3) Guest — Register

View: `app/views/auth/register.php` .

CSS: `public/css/register.css` (similar hard-coded path pattern possible).

Shared admin bundle: No.

4) Admin

Views: `app/views/admin/ad_*.php` .

Typical order:

1. Open document / head (varies slightly per file).
2. `include_once app/views/templates/admin/ad_admin_core_styles.php` (shared bundle above).
3. Page stylesheet: `public/css/admin/ad_<page>.css` (e.g. `ad_bookings.css` , `ad_dashboard.css`).
4. `include_once ad_header.php` and `ad_sidebar.php` .
5. Main content (often `.main-content`).
6. Close `</body></html>` in the same view.

Quirk: `ad_dashboard.php` may load Boxicons again after core styles (duplicate, harmless).

5) Manager & 6) HR (same UI shell)

Role: `manager` accounts use HR-area controllers/views for most office UI; notifications map manager → HR header/sidebar in `app/views/notifications/notification_page.php` .

Views: `app/views/hr/hr_*.php` (and related HR PHP files in that folder).

Typical order:

1. Set `$hrPageTitle` ; optional `$hrExtraCss` (paths under `public/css/`); optional `$hrHeadStylesheets` (full URLs, e.g. calendar CDN).
2. `include_once app/views/templates/hr/hr_layout_head.php` — opens `<html><head>` , sets title, then includes `ad_admin_core_styles.php` , then head sheets, then each extra CSS, then always `public/css/hr/hr-post-admin.css` , then `<body>` .
3. `include_once hr_header.php` and `hr_sidebar.php` .
4. Page content (usually `<main class="main-content">`).
5. `include_once hr_layout_close.php` — `</body></html>` .

Shared CSS: full core bundle + `hr-post-admin.css` + any page files listed in `$hrExtraCss` .

7) Client

Views: `app/views/client/c_*.php` (plus a few like `feedback_add.php`).

Typical order (example `c_dashboard.php`):

1. Set `$clientPageTitle` , `$clientExtraCss` (paths under `public/css/`), optional `$clientHeadStylesheets` .
2. `require_once app/views/templates/client/client_layout_head.php` — includes `ad_admin_core_styles.php` , optional head URLs, each extra CSS, then always `public/css/hr/hr-post-`

`admin.css` and `public/css/client/client-post-admin.css` , then opens `<body>` .

3. Optional PHP partials.

4. `require_once` `c_header.php` and `c_sidebar.php` under `app/views/templates/client/` .

5. Page body.

6. `require_once` `client_layout_close.php` .

Optional: `client_shell_open.php` bundles layout head + header + sidebar in one include for pages that use that pattern.

Shared CSS: core bundle + HR post layer + client post layer + per-page files from `$clientExtraCss` (dashboard often reuses `admin/ad_dashboard.css` for card layout).

8) Caretaker

Views: `app/views/caretaker/*.php` with templates in `app/views/templates/caretaker/` .

Note: Caretaker pages do *not* use `hr_layout_head` / `client_layout_head` . `ct_header.php` and `ct_sidebar.php` each carry their own `<head>` fragments; some page files then open another full HTML document. Treat this as legacy layout; consider refactoring later to one shell like Client/HR.

Typical `ct_dashboard.php` order:

1. `ct_header.php` — Font Awesome + `public/css/caretaker/ct_header.css` .

2. `ct_sidebar.php` — Boxicons + `ct_sidebar.css` + `common/sidebar-badges.css` .

3. Page file opens a new document and loads e.g. `ct_dashboard.css` .

Shared with admin design system: mainly `common/sidebar-badges.css` only; not `admin-ui.css` by default.

9) Notifications (all roles, one file)

File: `app/views/notifications/notification_page.php` .

Picks header/sidebar template by role; manager uses HR templates. Loads `public/css/admin/ad_notification.css` in each branch; Client/HR use their normal layout heads (so they still get the core bundle); Admin uses core + notification CSS; Caretaker uses a minimal head (Font Awesome + notification CSS only).

Summary table

Actor	Primary layout entry	Uses <code>ad_admin_core_styles.php</code>	Extra “always” layers
Landing	<code>landing/home.php</code>	No	<code>landing.css</code>
Login / Register	auth views	No	own CSS files

Actor	Primary layout entry	Uses ad_admin_core_styles.php	Extra “always” layers
Admin	admin views + core include	Yes	ad_*.css per page
HR / Manager	hr_layout_head.php	Yes	hr-post-admin.css + extras
Client	client_layout_head.php	Yes	hr-post-admin.css , client-post-admin.css , extras
Caretaker	ct_header.php / ct_sidebar.php	No (default)	ct_*.css + badges on sidebar

Design-system CSS folders under `public/css/system` , `layout` , `components` , `responsive` , and `base.css` were removed from the repo as unused by runtime views; documentation refers only to files that exist today.